

B.3.5. LIST / LIST OF

LIST Syntax

```

LIST [ list-qualifier ] OF type-or-ref           // uniform list
LIST [ list-qualifier ] { type-pattern }         // pattern list

list-qualifier (optional):
    length
    nullable

type-pattern:
    type-or-ref quantifier                       // unnamed item
    identifier [ qualifier ] : type-or-ref quantifier // named item

quantifier (optional):
    *                                           // zero or more
    +                                           // one or more
    { count }                                  // exactly count
    { min..max }                               // between min and max
    { min.. }                                  // at least min

qualifier (optional):
    tag

```

LIST and **LIST OF** declare an element that is encoded as a TLV [List](#). **LIST** and **LIST OF** declare the same fundamental type, but differ based on how the allowed types of their items are expressed.

LIST OF declares a list where all the items in the list are of the same fundamental type, or taken from the same set of possible types. This form of list is called a *uniform list*. Uniform lists are generally used to represent ordered collections of values where the tags differentiate the semantic meaning of the value.

LIST declares a *pattern list* where the types of the items in the list follow a particular pattern. In this form, the allowed type(s) for an item depends on its position in the array. Pattern lists are typically used to represent path-like constructs.

The overall pattern of types allowed in a pattern list is declared using a schema construct called a linear type pattern. The syntax and interpretation of linear type patterns for pattern lists are the same as those for pattern arrays (see [Section B.3.1.1, “Linear Type Patterns”](#)).

The [length](#) qualifier MAY be used to constraint the minimum and maximum number of items in the list. For a pattern list, the given length constraint SHALL be consistent with (i.e. fall within) the minimum and maximum number of items implied by the type pattern. In cases where the [length](#) qualifier places a narrower constraint on the length of a list than that implied by the type pattern, the [length](#) qualifier constraint takes precedence.